

TP 2 — Application de Gestion du Bureau Des Étudiants (BDE)

Alexis PICOT

TP 2 — Application de Gestion du Bureau Des Étudiants (BDE)

Contexte

Félicitations ! Vous venez d'être élus au Bureau Des Étudiants de votre école. Le président sortant vous fait un rapide topo :

« On gérait tout sur Excel jusqu'à maintenant, mais c'est devenu ingérable. On a 1500 étudiants, une dizaine d'événements par mois, des partenariats, une boutique de goodies, et on doit gérer les cotisations. Le trésorier précédent a démissionné parce qu'il passait ses nuits à faire des RECHERCHEV. On a besoin d'une vraie appli. »

Quand vous demandez des précisions, il vous répond :

« Commencez par gérer les adhérents et les événements, on verra le reste après. Ah, et l'appli doit être prête pour la semaine d'intégration dans 3 semaines. »

Vous voilà donc chargés de développer une application de gestion pour le BDE. Le cahier des charges est... disons... "évolutif".

Étape 1 — Gestion basique des adhérents et événements

Ce que le président vous demande

« Pour commencer, j'ai juste besoin de gérer la liste des adhérents et de créer des événements. Un adhérent, c'est simple : nom, prénom, email, promotion (genre "3ICS" ou "4ETI"), et s'il a payé sa cotisation ou pas. Pour les événements, on a besoin du nom, de la date, du lieu, du prix, et de la liste des inscrits. »

Votre mission

Vous devez créer une application console qui permet de :

1. **Ajouter un adhérent** au BDE
 - Saisir ses informations (nom, prénom, email, promotion)
 - Par défaut, la cotisation n'est pas payée
2. **Enregistrer le paiement** de la cotisation d'un adhérent
 - Rechercher l'adhérent par son email
 - Marquer sa cotisation comme payée
3. **Créer un événement**
 - Définir le nom, la date, le lieu et le prix
4. **Inscrire un adhérent à un événement**

- Vérifier que l'adhérent existe
- L'ajouter à la liste des participants

5. **Afficher la liste des participants** à un événement

6. **Afficher le récapitulatif** d'un adhérent (ses infos + les événements auxquels il est inscrit)

Contraintes techniques

- L'application fonctionne en mode console avec un menu textuel
- Les données sont stockées en mémoire (pas de base de données)
- Vous devez pouvoir avoir plusieurs événements et plusieurs adhérents

Exemple d'interaction attendue

=== BDE Manager ===

1. Ajouter un adhérent
2. Payer une cotisation
3. Créer un événement
4. Inscrire à un événement
5. Afficher participants d'un événement
6. Afficher fiche adhérent
7. Quitter

Votre choix : 1

Nom : Dupont

Prénom : Marie

Email : marie.dupont@cpe.fr

Promotion : 3ICS

Adhérent ajouté avec succès !

Votre choix : 3

Nom de l'événement : Soirée d'intégration

Date (JJ/MM/AAAA) : 15/09/2026

Lieu : Le Ninkasi

Prix : 15

Événement créé avec succès !

Votre choix : 4

Email de l'adhérent : marie.dupont@cpe.fr

Nom de l'événement : Soirée d'intégration

Inscription réussie !

Questions à vous poser avant de coder

- Comment allez-vous stocker les adhérents ? Les événements ?
- Comment faire le lien entre un adhérent et les événements auxquels il participe ?
- Que se passe-t-il si on essaie d'inscrire quelqu'un qui n'existe pas ? **Renvoyer null n'est pas une réponse acceptable.** Pensez aux **exceptions**.
- Comment identifier un adhérent de manière unique ? Un simple email ? Un identifiant auto-incrémenté ?
- Si deux adhérents ont le même email, que se passe-t-il ?

Utilisation de l'IA :

A cette étape, si vous avez bien créé des méthodes plutôt que de mettre votre comportement dans le menu , vous devriez vous retrouver avec des méthodes `creerAdherent()`, `payerCotisation()`, `creerEvenement()`, etc.

Vous pouvez demander à l'IA de vous générer un bloc de code pour faciliter vos tests, car sinon à chaque fois vous devez recréer les adhérents etc. Comme vous ne savez pas encore comment lire depuis une BDD (du moins dans ce cours, ne soyez pas trop tentés de le faire maintenant vous risqueriez d'y passer 3h, demandez moi si toutefois vous souhaiteriez vous lancer quand même)

Le code généré par l'IA pourrait ressembler à ça :

```
public void generateData() {
    creerAdherent("Dupont", "Marie", "marie.dupont@cpe.fr", "3ICS");
    creerAdherent("Martin", "Thomas", "thomas.martin@cpe.fr", "3ICS");
    creerAdherent("Bernard", "Sophie", "sophie.bernard@cpe.fr", "4ETI");
    creerAdherent("Lefevre", "Antoine", "antoine.lefevre@cpe.fr", "3ICS");
    creerAdherent("Moreau", "Laura", "laura.moreau@external.fr", "External");
    creerAdherent("Dubois", "Pierre", "pierre.dubois@cpe.fr", "Bureau");

    payerCotisation("marie.dupont@cpe.fr");
    payerCotisation("thomas.martin@cpe.fr");
    payerCotisation("sophie.bernard@cpe.fr");

    creerEvenement("Soirée d'intégration", "15/09/2026", "Le Ninkasi", 15);
    creerEvenement("Gala du BDE", "10/11/2026", "Amphithéâtre CPE", 20);
    creerEvenement("Tournoi de foot", "22/10/2026", "Stade Gerland", 10);

    inscrireEvenement("marie.dupont@cpe.fr", "Soirée d'intégration");
    inscrireEvenement("thomas.martin@cpe.fr", "Soirée d'intégration");
    inscrireEvenement("sophie.bernard@cpe.fr", "Tournoi de foot");
}
```

Correction Étape 1

À ce stade, vous devriez avoir des classes simples mais bien structurées.

Concept : static pour les identifiants — On utilise un attribut `static` pour générer automatiquement un identifiant unique pour chaque adhérent. Le compteur appartient à la **classe** (partagé entre toutes les instances), pas à une instance particulière.

```
public class Adherent {
    private static int prochainId = 1; // static : partagé entre toutes les instances

    private int id;
    private String nom;
    private String prenom;
    private String email;
    private String promotion;
    private boolean cotisationPayee;

    public Adherent(String nom, String prenom, String email, String promotion) {
        this.id = prochainId++; // chaque adhérent a un ID unique
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
    }
}
```

```

        this.promotion = promotion;
        this.cotisationPayee = false;
    }

    public int getId() { return id; }
    public String getEmail() { return email; }
    public String getNom() { return nom; }
    public String getPrenom() { return prenom; }
    public boolean isCotisationPayee() { return cotisationPayee; }

    public void payerCotisation() {
        this.cotisationPayee = true;
    }

    @Override
    public String toString() {
        String statut = cotisationPayee ? "Cotisation payée" : "Cotisation non payée";
        return "[" + id + "]" + prenom + " " + nom + " (" + promotion + ") - " + statut;
    }
}

```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class Evenement {
    private String nom;
    private LocalDate date;
    private String lieu;
    private double prix;
    private List<Adherent> participants;

    public Evenement(String nom, LocalDate date, String lieu, double prix) {
        this.nom = nom;
        this.date = date;
        this.lieu = lieu;
        this.prix = prix;
        this.participants = new ArrayList<>();
    }

    public String getNom() { return nom; }
    public double getPrix() { return prix; }

    public void inscrire(Adherent adherent) {
        if (!participants.contains(adherent)) {
            participants.add(adherent);
        }
    }

    public List<Adherent> getParticipants() {
        return new ArrayList<>(participants);
    }

    @Override

```

```

    public String toString() {
        return nom + " - " + date + " @ " + lieu + " (" + prix + "€";
    }
}

import java.util.*;

public class BDE {
    private List<Adherent> adherents;
    private List<Evenement> evenements;

    public BDE() {
        this.adherents = new ArrayList<>();
        this.evenements = new ArrayList<>();
    }

    public void ajouterAdherent(Adherent adherent) {
        adherents.add(adherent);
    }

    public void creerEvenement(Evenement evenement) {
        evenements.add(evenement);
    }

    public Adherent trouverAdherent(String email) {
        return adherents.stream()
            .filter(a -> a.getEmail().equals(email))
            .findFirst()
            .orElseThrow(() -> new AdherentIntrouvableException(email));
    }

    public Evenement trouverEvenement(String nom) {
        return evenements.stream()
            .filter(e -> e.getNom().equals(nom))
            .findFirst()
            .orElseThrow(() -> new EvenementIntrouvableException(nom));
    }
}

```

Concept : Exceptions personnalisées — Plutôt que de renvoyer null (ce qui cause des NullPointerException mystérieuses), on lance des exceptions parlantes. L'appelant sait exactement ce qui s'est passé.

```

public class AdherentIntrouvableException extends RuntimeException {
    public AdherentIntrouvableException(String email) {
        super("Aucun adhérent trouvé avec l'email : " + email);
    }
}

public class EvenementIntrouvableException extends RuntimeException {
    public EvenementIntrouvableException(String nom) {
        super("Aucun événement trouvé avec le nom : " + nom);
    }
}

```

Points importants : - L'Adhérent sait payer sa cotisation - L'Événement gère sa propre liste de participants
- Le BDE orchestre mais ne contient pas toute la logique - Les exceptions personnalisées remplacent les `return null` dangereux

Étape 2 — Le trésorier s'en mêle

Vous avez à peine terminé l'étape 1 que le trésorier débarque dans votre local :

« Super votre truc ! Mais j'ai besoin de gérer l'argent. Les cotisations, c'est 30€ pour les étudiants CPE et 50€ pour les extérieurs. Et pour les événements, les adhérents ont une réduction de 20%. Ah, et les membres du bureau (président, trésorier, secrétaire, etc.) ont tout gratuit. »

Puis il ajoute, l'air de rien :

« Et il faudrait que je puisse voir le solde du BDE. On reçoit l'argent des cotisations et des inscriptions, et on a des dépenses pour organiser les événements. Je veux savoir combien on a en caisse à tout moment. »

Nouvelles fonctionnalités à implémenter

1. Différencier les types d'adhérents

- Étudiant CPE : cotisation à 30€
- Extérieur : cotisation à 50€
- Membre du bureau : cotisation gratuite + événements gratuits

2. Appliquer les réductions sur les événements

- Adhérent avec cotisation payée : -20% sur le prix des événements
- Membre du bureau : gratuit
- Non-adhérent (cotisation non payée) : plein tarif

3. Calculer le prix d'inscription à un événement pour un adhérent donné

4. Gérer les finances du BDE

- Enregistrer les recettes (cotisations + inscriptions événements)
- Enregistrer les dépenses (coût d'organisation des événements)
- Afficher le solde actuel

5. Ajouter un coût d'organisation aux événements

- Chaque événement a un budget prévisionnel
- Ce budget est une dépense pour le BDE

Réflexions imposées

Avant de modifier votre code, répondez à ces questions :

- Où allez-vous mettre la logique de calcul du prix de la cotisation ?
- Où allez-vous mettre la logique de calcul du prix d'un événement pour un adhérent ?
- Comment allez-vous différencier un étudiant CPE d'un extérieur d'un membre du bureau ?
- Si demain on ajoute un nouveau type d'adhérent (ancien élève, par exemple), que devrez-vous modifier ?

Attention !

Si vous êtes tentés d'écrire quelque chose comme :

```

if (adherent.getType().equals("CPE")) {
    prix = 30;
} else if (adherent.getType().equals("EXTERIEUR")) {
    prix = 50;
} else if (adherent.getType().equals("BUREAU")) {
    prix = 0;
}

```

STOP ! Appelez l'enseignant. Ce code va devenir un cauchemar à maintenir.

Correction Étape 2

C'est le moment d'introduire l'**héritage** pour les types d'adhérents :

```

public abstract class Adherent {
    private static int prochainId = 1;

    private int id;
    private String nom;
    private String prenom;
    private String email;
    private String promotion;
    private boolean cotisationPayee;

    public Adherent(String nom, String prenom, String email, String promotion) {
        this.id = prochainId++;
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.promotion = promotion;
        this.cotisationPayee = false;
    }

    // Chaque type d'adhérent définit son prix de cotisation
    public abstract double getPrixCotisation();

    // Chaque type d'adhérent définit sa réduction sur les événements
    public abstract double getReductionEvenement();

    public boolean isCotisationPayee() { return cotisationPayee; }
    public String getEmail() { return email; }

    public double payerCotisation() {
        this.cotisationPayee = true;
        return getPrixCotisation(); // Retourne le montant pour la comptabilité
    }

    // Calcule le prix d'un événement pour cet adhérent
    public double calculerPrixEvenement(double prixBase) {
        if (!cotisationPayee) {
            return prixBase; // Plein tarif si pas adhérent
        }
        return prixBase * (1 - getReductionEvenement());
    }
}

```

```

}

public class EtudiantCPE extends Adherent {
    public EtudiantCPE(String nom, String prenom, String email, String promotion) {
        super(nom, prenom, email, promotion);
    }

    @Override
    public double getPrixCotisation() { return 30.0; }

    @Override
    public double getReductionEvenement() { return 0.20; } // -20%
}

public class Exterieur extends Adherent {
    public Exterieur(String nom, String prenom, String email, String promotion) {
        super(nom, prenom, email, promotion);
    }

    @Override
    public double getPrixCotisation() { return 50.0; }

    @Override
    public double getReductionEvenement() { return 0.20; } // -20%
}

public class MembreBureau extends Adherent {
    private String role; // Président, Trésorier, etc.

    public MembreBureau(String nom, String prenom, String email, String role) {
        super(nom, prenom, email, "Bureau");
        this.role = role;
    }

    @Override
    public double getPrixCotisation() { return 0.0; } // Gratuit

    @Override
    public double getReductionEvenement() { return 1.0; } // 100% = gratuit
}

public class ComptaBDE {
    private double solde;
    private List<Transaction> historique;

    public ComptaBDE() {
        this.solde = 0;
        this.historique = new ArrayList<>();
    }

    public void enregistrerRecette(double montant, String description) {
        solde += montant;
        historique.add(new Transaction(montant, description, true));
    }
}

```



```

public void enregistrerDepense(double montant, String description) {
    solde -= montant;
    historique.add(new Transaction(montant, description, false));
}

public double getSolde() { return solde; }
}

```

Concept : record — La classe `Transaction` ne fait que stocker des données immuables. C'est le cas parfait pour un **record** Java (Java 14+) : pas de setter, `equals`/`hashCode`/`toString` générés automatiquement.

```

// Un record remplace une classe entière avec constructeur, getters, equals, hashCode, toString
public record Transaction(double montant, String description, boolean estRecette) {
    // Compact constructor pour validation
    public Transaction {
        if (montant < 0) throw new IllegalArgumentException("Le montant ne peut pas être négatif");
    }
}

```

Concept : SOLID en action — Ce code applique plusieurs principes SOLID : - **SRP** : Chaque classe a une responsabilité unique (l'adhérent connaît son prix, la compta gère les finances) - **OCP** : Pour ajouter un nouveau type d'adhérent, on crée une sous-classe sans modifier le code existant - **LSP** : Un `MembreBureau` peut remplacer un `Adherent` partout où un `Adherent` est attendu

On peut aussi enrichir la gestion des erreurs avec des exceptions dédiées :

```

public class CotisationDejaPayeeException extends RuntimeException {
    public CotisationDejaPayeeException(Adherent adherent) {
        super(adherent.getPrenom() + " " + adherent.getNom() + " a déjà payé sa cotisation");
    }
}

public class InscriptionDupliqueeException extends RuntimeException {
    public InscriptionDupliqueeException(Adherent adherent, Evenement evenement) {
        super(adherent.getPrenom() + " est déjà inscrit à " + evenement.getNom());
    }
}

```

Points clés : - Le **polymorphisme** évite les `switch/if` sur le type d'adhérent - Chaque adhérent **sait** calculer son prix - Pour ajouter "Ancien élève", il suffit de créer une nouvelle sous-classe - Le **record** `Transaction` remplace avantageusement une classe complète pour des données immuables - Le compteur **static** génère des IDs uniques sans passer par une base de données

Pause Refactoring — Avant de continuer

Avant de passer à l'étape 3, prenez 15 minutes pour examiner votre code :

Check-list SOLID

- ☐ **SRP** : Votre classe `BDE` fait-elle trop de choses ? Si elle dépasse 200 lignes, c'était probablement le cas.
- ☐ **OCP** : Si on vous demandait d'ajouter un type "Ancien élève" avec cotisation à 40€, combien de fichiers devriez-vous modifier ? (Réponse idéale : 1 seul, la nouvelle classe)
- ☐ **LSP** : Partout où vous utilisez `Adherent`, est-ce que ça marcherait si c'était un `MembreBureau` ? Un `Exterieur` ?

Check-list gestion d'erreurs

- ☐ Créez-vous des **exceptions personnalisées** (`AdherentIntrouvableException`, `CotisationDejaPayeeException`, `InscriptionDupliqueeException`) ?
- ☐ Votre méthode `inscrire()` vérifie-t-elle les cas d'erreur (adhérent déjà inscrit, événement complet) ?
- ☐ Avez-vous un `try/catch` dans votre menu console pour gérer les erreurs de saisie ?

Check-list bonnes pratiques

- ☐ Utilisez-vous un **record** pour `Transaction` ?
- ☐ Vos adhérents ont-ils un ID généré automatiquement (attribut `static`) ?
- ☐ Votre méthode `payerCotisation()` vérifie-t-elle que la cotisation n'est pas déjà payée ?

Si vous répondez “non” à plus de 3 cases, prenez le temps de refactorer **maintenant**. Ça sera beaucoup plus douloureux après.

Étape 3 — Les clubs entrent en jeu

La secrétaire générale vous interpelle pendant la pause :

« J'ai oublié de vous dire, on a aussi des clubs ! Le club photo, le club musique, le club robotique... Chaque club a un président, un budget propre, et peut organiser ses propres événements. Les membres d'un club ont accès gratuit aux événements de leur club. »

Elle réfléchit quelques secondes puis ajoute :

« Ah et un adhérent peut être membre de plusieurs clubs. Et un club peut co-organiser un événement avec le BDE ou avec un autre club. Dans ce cas, les frais sont partagés et les membres de tous les clubs organisateurs ont l'événement gratuit. »

Nouvelles fonctionnalités

1. **Créer des clubs**
 - Chaque club a un nom, un président (qui doit être adhérent), et un budget
2. **Gérer les membres des clubs**
 - Un adhérent peut rejoindre un ou plusieurs clubs
 - Chaque club a sa propre liste de membres
3. **Événements de club**
 - Un club peut créer des événements
 - Les membres du club ont ces événements gratuits
 - Le coût est déduit du budget du club (pas du BDE)
4. **Co-organisation d'événements**
 - Un événement peut être organisé par le BDE seul, un club seul, ou plusieurs entités ensemble
 - Les frais sont répartis entre les organisateurs
 - Les membres de toutes les entités organisatrices ont l'événement gratuit
5. **Afficher les événements auxquels un adhérent a droit gratuitement**
 - Événements du BDE (s'il est membre du bureau)
 - Événements de ses clubs
 - Événements co-organisés par ses clubs

Le piège qui vous attend

Si vous avez un attribut `organisateur` de type `String` dans votre classe événement... vous allez souffrir.

Si vous avez dupliqué du code entre “événement BDE” et “événement club”... vous allez souffrir.

Si vous n'avez pas prévu qu'un événement puisse avoir plusieurs organisateurs... vous allez souffrir.

C'est le moment de repenser votre conception.

Questions de conception

- Comment représenter le fait qu'un événement peut être organisé par différentes entités (BDE, clubs) ?
- Comment éviter de dupliquer le code de gestion des événements ?
- Comment gérer élégamment le fait qu'un adhérent peut avoir des "droits" venant de différentes sources ?

Correction Étape 3

C'est le moment d'introduire les **interfaces** ! Un organisateur peut être le BDE ou un Club :

```
public interface Organisateur {
    String getNom();
    double getBudget();
    void debiterBudget(double montant);
    boolean estMembre(Adherent adherent);
}
```

```
public class BDE implements Organisateur {
    private List<Adherent> adherents;
    private List<Evenement> evenements;
    private ComptaBDE compta;
    private List<MembreBureau> bureau;

    @Override
    public String getNom() { return "BDE"; }

    @Override
    public double getBudget() { return compta.getSolde(); }

    @Override
    public void debiterBudget(double montant) {
        compta.enregistrerDepense(montant, "Organisation événement");
    }

    @Override
    public boolean estMembre(Adherent adherent) {
        return adherent instanceof MembreBureau;
    }
}
```

```
public class Club implements Organisateur {
    private String nom;
    private Adherent president;
    private double budget;
    private List<Adherent> membres;

    public Club(String nom, Adherent president, double budgetInitial) {
        this.nom = nom;
        this.president = president;
        this.budget = budgetInitial;
        this.membres = new ArrayList<>();
        this.membres.add(president);
    }
}
```

```

@Override
public String getNom() { return nom; }

@Override
public double getBudget() { return budget; }

@Override
public void debiterBudget(double montant) {
    this.budget -= montant;
}

@Override
public boolean estMembre(Adherent adherent) {
    return membres.contains(adherent);
}

public void ajouterMembre(Adherent adherent) {
    if (!membres.contains(adherent)) {
        membres.add(adherent);
    }
}
}

import java.util.*;

public class Evenement {
    private String nom;
    private LocalDate date;
    private String lieu;
    private double prixBase;
    private double coutOrganisation;
    private List<Organisateur> organisateurs; // Peut avoir plusieurs !
    private List<Adherent> participants;

    public Evenement(String nom, LocalDate date, String lieu,
                     double prixBase, double coutOrganisation) {
        this.nom = nom;
        this.date = date;
        this.lieu = lieu;
        this.prixBase = prixBase;
        this.coutOrganisation = coutOrganisation;
        this.organisateurs = new ArrayList<>();
        this.participants = new ArrayList<>();
    }

    public void ajouterOrganisateur(Organisateur org) {
        organisateurs.add(org);
    }

    // Répartit les frais entre tous les organisateurs
    public void repartirFrais() {
        double partChacun = coutOrganisation / organisateurs.size();
        for (Organisateur org : organisateurs) {

```

```

        org.debiterBudget(partChacun);
    }
}

// Vérifie si l'adhérent a l'événement gratuit
public boolean estGratuitPour(Adherent adherent) {
    for (Organisateur org : organisateurs) {
        if (org.estMembre(adherent)) {
            return true;
        }
    }
    return false;
}

// Calcule le prix pour un adhérent
public double calculerPrixPour(Adherent adherent) {
    if (estGratuitPour(adherent)) {
        return 0;
    }
    return adherent.calculerPrixEvenement(prixBase);
}
}

```

Points clés : - L'interface `Organisateur` permet de traiter BDE et Club de la même façon - Un événement peut avoir **plusieurs** organisateurs (co-organisation) - Le polymorphisme vérifie automatiquement l'appartenance

Étape 4 — La boutique de goodies

Le vice-président communication arrive avec une nouvelle demande :

« On a une boutique de goodies ! On vend des sweats, des t-shirts, des tote bags... Chaque article a un nom, un prix, et un stock. Les adhérents ont 10% de réduction, les membres du bureau 50%. Et certains articles sont réservés aux adhérents. »

Puis, avec un grand sourire :

« Ah et on fait des packs ! Genre le “Pack Intégration” avec un sweat + un tote bag + une place pour la soirée d'intégration, le tout avec 30% de réduction. Un pack peut contenir des articles ET des places pour des événements. »

Nouvelles fonctionnalités

1. **Gérer un catalogue d'articles**
 - Créer des articles avec nom, prix, stock
 - Certains articles sont réservés aux adhérents
2. **Vendre un article**
 - Vérifier le stock
 - Appliquer la réduction selon le statut de l'acheteur
 - Décrémenter le stock
 - Enregistrer la recette
3. **Créer des packs**
 - Un pack contient plusieurs éléments (articles et/ou places événements)
 - Un pack a un prix global avec réduction

4. Acheter un pack

- Tout le contenu du pack doit être disponible
- Appliquer les règles spécifiques de chaque élément
- Gérer le stock et les inscriptions

Réflexion architecturale

Un pack contient des choses très différentes : des objets physiques (goodies) et des choses immatérielles (inscriptions à des événements). Pourtant, on veut pouvoir les manipuler de manière uniforme (calculer un prix, vérifier la disponibilité, “consommer” l’achat).

Comment représenter cette situation en POO ?

Indice : C’est le moment de découvrir (ou redécouvrir) la notion d’**interface**. Si vous n’y arrivez pas, appelez l’enseignant.

Correction Étape 4

Une **interface** Achetable permet de traiter articles et places événements uniformément :

```
public interface Achetable {
    String getNom();
    double getPrix();
    boolean estDisponible();
    void consommer(); // Décrémente stock ou inscrit à l'événement
}

public class Article implements Achetable {
    private String nom;
    private double prix;
    private int stock;
    private boolean reserveAuxAdherents;

    public Article(String nom, double prix, int stock, boolean reserveAuxAdherents) {
        this.nom = nom;
        this.prix = prix;
        this.stock = stock;
        this.reserveAuxAdherents = reserveAuxAdherents;
    }

    @Override
    public String getNom() { return nom; }

    @Override
    public double getPrix() { return prix; }

    @Override
    public boolean estDisponible() { return stock > 0; }

    @Override
    public void consommer() {
        if (stock > 0) stock--;
    }

    public boolean isReserveAuxAdherents() { return reserveAuxAdherents; }
}
```

```

public class PlaceEvenement implements Achetable {
    private Evenement evenement;
    private Adherent pourAdherent; // Pour qui on achète la place

    public PlaceEvenement(Evenement evenement, Adherent pourAdherent) {
        this.evenement = evenement;
        this.pourAdherent = pourAdherent;
    }

    @Override
    public String getNom() { return "Place: " + evenement.getNom(); }

    @Override
    public double getPrix() {
        return evenement.calculerPrixPour(pourAdherent);
    }

    @Override
    public boolean estDisponible() {
        return !evenement.getParticipants().contains(pourAdherent);
    }

    @Override
    public void consommer() {
        evenement.inscrire(pourAdherent);
    }
}

```

```

import java.util.*;

public class Pack implements Achetable {
    private String nom;
    private List<Achetable> elements;
    private double reduction; // ex: 0.30 pour -30%

    public Pack(String nom, double reduction) {
        this.nom = nom;
        this.elements = new ArrayList<>();
        this.reduction = reduction;
    }

    public void ajouter(Achetable element) {
        elements.add(element);
    }

    @Override
    public String getNom() { return nom; }

    @Override
    public double getPrix() {
        double total = elements.stream()
            .mapToDouble(Achetable::getPrix)
            .sum();
        return total * (1 - reduction);
    }
}

```

```

    }

    @Override
    public boolean estDisponible() {
        // Tous les éléments doivent être disponibles
        return elements.stream().allMatch(Achetable::estDisponible);
    }

    @Override
    public void consommer() {
        // Consomme tous les éléments
        elements.forEach(Achetable::consommer);
    }
}

public class Boutique {
    private List<Article> catalogue;
    private ComptaBDE compta;

    public double vendre(Achetable achat, Adherent acheteur) {
        if (!achat.estDisponible()) {
            throw new ArticleIndisponibleException(achat.getNom());
        }

        double prix = achat.getPrix();

        // Pattern matching instanceof (Java 16+)
        // Plus lisible que instanceof + cast séparés
        if (achat instanceof Article article && acheteur.isCotisationPayee()) {
            if (acheteur instanceof MembreBureau mb) {
                prix *= 0.5; // -50% pour le bureau
            } else {
                prix *= 0.9; // -10% pour les adhérents
            }

            if (article.isReserveAuxAdherents() && !acheteur.isCotisationPayee()) {
                throw new ArticleReserveException(article.getNom());
            }
        }

        achat.consommer();
        compta.enregistrerRecette(prix, "Vente: " + achat.getNom());

        return prix;
    }
}

```

Points clés : - L'interface `Achetable` unifie articles, places et packs - Le `Pack` implémente aussi `Achetable` (composition récursive possible) - Une seule méthode `vendre()` gère tous les types d'achats - Le **pattern matching instanceof** (Java 16+) évite les casts manuels : `if (achat instanceof Article article)` équivaut à `if (achat instanceof Article) { Article article = (Article) achat; ... }` mais en plus sûr et lisible - Les **exceptions personnalisées** (`ArticleIndisponibleException`, `ArticleReserveException`) remplacent les `RuntimeException` génériques

Étape 5 — Les partenariats

Le président revient avec des nouvelles :

« On a signé des partenariats ! Certaines entreprises nous donnent de l'argent en échange de visibilité. Mais c'est compliqué... Il y a plusieurs niveaux : Bronze (500€), Silver (1000€), Gold (2000€), Platinum (5000€). Chaque niveau donne des avantages différents. »

Il sort une feuille froissée de sa poche :

« Bronze : logo sur nos affiches. Silver : pareil + stand aux événements. Gold : pareil + publication sur nos réseaux. Platinum : pareil + naming d'un événement + places gratuites pour leurs employés. »

Et comme si ça ne suffisait pas :

« Ah et les partenaires peuvent offrir des réductions à nos adhérents. Genre -15% chez le partenaire pizza. Il faut qu'on puisse générer une “carte avantages” pour chaque adhérent avec tous les avantages auxquels il a droit. »

Nouvelles fonctionnalités

1. **Gérer les partenaires**
 - Créer un partenaire avec nom, niveau de partenariat
 - Enregistrer le versement du partenariat (recette pour le BDE)
2. **Avantages par niveau**
 - Chaque niveau donne des droits spécifiques
 - Les niveaux supérieurs cumulent les avantages des niveaux inférieurs
3. **Réductions partenaires**
 - Un partenaire peut offrir une réduction aux adhérents
 - La réduction a une description et un pourcentage
4. **Générer la carte avantages d'un adhérent**
 - Liste des réductions partenaires
 - Événements gratuits (bureau, clubs)
 - Réductions sur la boutique

Le piège classique

Vous allez être tentés de faire :

```
if (niveau.equals("BRONZE")) {  
    // trucs bronze  
} else if (niveau.equals("SILVER")) {  
    // trucs bronze + trucs silver  
} else if (niveau.equals("GOLD")) {  
    // trucs bronze + trucs silver + trucs gold  
} else if (niveau.equals("PLATINUM")) {  
    // tout le reste  
}
```

Ce code est : - Dupliqué (on répète les avantages des niveaux inférieurs) - Fragile (si on ajoute un niveau “Diamond”, il faut tout modifier) - Illisible (le “else if” va faire 50 lignes)

Comment faire mieux ?

Correction Étape 5

Une **énumération** avec héritage de comportement pour les niveaux de partenariat :

```

import java.util.*;

public enum NiveauPartenariat {
    BRONZE(500, List.of("Logo sur affiches")),
    SILVER(1000, List.of("Logo sur affiches", "Stand aux événements")),
    GOLD(2000, List.of("Logo sur affiches", "Stand aux événements",
        "Publication réseaux sociaux")),
    PLATINUM(5000, List.of("Logo sur affiches", "Stand aux événements",
        "Publication réseaux sociaux", "Naming événement",
        "Places gratuites employés"));

    private final double montant;
    private final List<String> avantages;

    NiveauPartenariat(double montant, List<String> avantages) {
        this.montant = montant;
        this.avantages = avantages;
    }

    public double getMontant() { return montant; }
    public List<String> getAvantages() { return avantages; }

    // Vérifie si ce niveau a un avantage spécifique
    public boolean aAvantage(String avantage) {
        return avantages.contains(avantage);
    }
}

import java.util.*;

public class Partenaire {
    private String nom;
    private NiveauPartenariat niveau;
    private List<ReductionPartenaire> reductions;

    public Partenaire(String nom, NiveauPartenariat niveau) {
        this.nom = nom;
        this.niveau = niveau;
        this.reductions = new ArrayList<>();
    }

    public String getNom() { return nom; }
    public NiveauPartenariat getNiveau() { return niveau; }

    public void ajouterReduction(String description, double pourcentage) {
        reductions.add(new ReductionPartenaire(this, description, pourcentage));
    }

    public List<ReductionPartenaire> getReductions() {
        return new ArrayList<>(reductions);
    }

    // Les avantages sont délégués à l'enum
    public List<String> getAvantages() {

```

```

        return niveau.getAvantages();
    }

    public boolean peutAvoirNaming() {
        return niveau.aAvantage("Naming événement");
    }
}

public class ReductionPartenaire {
    private Partenaire partenaire;
    private String description;
    private double pourcentage;

    public ReductionPartenaire(Partenaire partenaire, String description,
                               double pourcentage) {
        this.partenaire = partenaire;
        this.description = description;
        this.pourcentage = pourcentage;
    }

    public String getDescription() {
        return partenaire.getNom() + ": " + description +
            " (-" + (pourcentage * 100) + "%)";
    }
}

import java.util.*;

public class CarteAvantages {
    private Adherent adherent;
    private List<String> avantages;

    public CarteAvantages(Adherent adherent, BDE bde) {
        this.adherent = adherent;
        this.avantages = new ArrayList<>();
        generer(bde);
    }

    private void generer(BDE bde) {
        // 1. Réductions partenaires (pour tous les adhérents à jour)
        if (adherent.isCotisationPayee()) {
            for (Partenaire p : bde.getPartenaires()) {
                for (ReductionPartenaire r : p.getReductions()) {
                    avantages.add(r.getDescription());
                }
            }
        }

        // 2. Événements gratuits (membre bureau)
        if (adherent instanceof MembreBureau) {
            avantages.add("Tous les événements BDE gratuits");
        }

        // 3. Événements des clubs
    }
}

```

```

    for (Club club : bde.getClubs()) {
        if (club.estMembre(adherent)) {
            avantages.add("Événements " + club.getNom() + " gratuits");
        }
    }

    // 4. Réductions boutique
    if (adherent.isCotisationPayee()) {
        if (adherent instanceof MembreBureau) {
            avantages.add("Boutique: -50%");
        } else {
            avantages.add("Boutique: -10%");
        }
    }
}

public void afficher() {
    System.out.println("=== Carte Avantages ===");
    System.out.println("Adhérent: " + adherent);
    for (String avantage : avantages) {
        System.out.println("    • " + avantage);
    }
}
}

```

Points clés : - L'enum NiveauPartenariat encapsule les avantages par niveau - Pas de chaîne de if/else, les avantages sont dans l'enum - Pour ajouter "Diamond", il suffit d'ajouter une ligne à l'enum

Attendus fonctionnels (récapitulatif)

À la fin du TP, votre application doit permettre de :

Gestion des adhérents

- ☐ Créer un adhérent (CPE, extérieur, ou membre du bureau)
- ☐ Enregistrer le paiement d'une cotisation
- ☐ Afficher la fiche complète d'un adhérent
- ☐ Générer la carte avantages d'un adhérent

Gestion des événements

- ☐ Créer un événement (BDE, club, ou co-organisé)
- ☐ Calculer le prix pour un adhérent donné
- ☐ Inscrire un adhérent
- ☐ Afficher les participants

Gestion des clubs

- ☐ Créer un club avec son président
- ☐ Ajouter/retirer des membres
- ☐ Gérer le budget du club

Gestion de la boutique

- ☐ Gérer un catalogue d'articles
- ☐ Créer des packs (articles + événements)
- ☐ Effectuer des ventes avec les réductions appropriées

Gestion des partenaires

- ☐ Enregistrer des partenaires avec leur niveau
- ☐ Gérer les réductions partenaires

Gestion financière

- ☐ Suivre les recettes et dépenses
 - ☐ Afficher le solde du BDE
 - ☐ Afficher le solde de chaque club
-

Pièges classiques dans lesquels vous allez tomber

Piège 1 : Le switch géant sur le type d'adhérent

Vous allez probablement écrire quelque chose comme :

```
public double calculerPrixCotisation(Adherent a) {
    switch(a.getType()) {
        case "CPE": return 30;
        case "EXTERIEUR": return 50;
        case "BUREAU": return 0;
        default: throw new RuntimeException("Type inconnu");
    }
}
```

Pourquoi c'est un problème ? - Chaque fois qu'on ajoute un type, il faut modifier ce switch - Ce switch va se dupliquer partout (prix cotisation, réduction événement, réduction boutique...) - Le compilateur ne vous préviendra pas si vous oubliez un cas

Indice : C'est l'occasion de mettre en pratique le **polymorphisme**. Chaque type d'adhérent devrait savoir calculer son propre prix de cotisation.

Piège 2 : Le modèle anémique

Vous allez probablement créer des classes comme :

```
public class Adherent {
    private String nom;
    private String prenom;
    private String email;
    private String type;
    private boolean cotisationPayee;

    // Que des getters et setters...
}
```

Et ensuite une classe "utilitaire" :

```
public class GestionAdherents {
    public static double calculerPrixCotisation(Adherent a) { ... }
}
```

```

public static double calculerPrixEvenement(Adherent a, Evenement e) { ... }
public static boolean peutAcheterArticle(Adherent a, Article art) { ... }
}

```

Pourquoi c'est un problème ? - L'adhérent ne "sait" rien faire, il est juste un sac de données - Toute la logique est dispersée dans des classes utilitaires - Impossible de profiter du polymorphisme

À retenir : Un objet doit encapsuler ses données ET ses comportements. Un adhérent devrait savoir calculer ce qu'il doit payer.

Piège 3 : L'organisateur en String

```

public class Evenement {
    private String organisateur; // "BDE" ou "Club Photo" ou ...
}

```

Pourquoi c'est un problème ? - Comment gérer un événement co-organisé ? - Comment accéder au budget de l'organisateur pour déduire les frais ? - Comment vérifier si un adhérent est membre de l'organisateur ?

Indice : Pensez aux **interfaces**. Un organisateur, c'est « quelque chose qui peut organiser des événements et qui a un budget ».

Piège 4 : La duplication Article / Place événement

Pour les packs, vous allez peut-être faire :

```

public class Pack {
    private List<Article> articles;
    private List<Evenement> evenements;

    public double calculerPrix() {
        double total = 0;
        for (Article a : articles) {
            total += a.getPrix();
        }
        for (Evenement e : evenements) {
            total += e.getPrix();
        }
        return total * 0.7; // -30%
    }

    public boolean estDisponible() {
        for (Article a : articles) {
            if (a.getStock() <= 0) return false;
        }
        for (Evenement e : evenements) {
            if (e.getPlacesRestantes() <= 0) return false;
        }
        return true;
    }
}

```

Pourquoi c'est un problème ? - Code dupliqué entre les deux boucles - Si on ajoute un nouveau type d'élément (ex: abonnement), il faut tout modifier - Le pack "sait" comment fonctionnent les articles et les événements

Indice : Pensez **interface** ! Un « élément achetable » sait donner son prix et dire s'il est disponible.

Piège 5 : Les niveaux de partenariat en chaîne de if

```
public List<String> getAvantages(Partenaire p) {
    List<String> avantages = new ArrayList<>();

    if (p.getNiveau().equals("BRONZE") || p.getNiveau().equals("SILVER")
        || p.getNiveau().equals("GOLD") || p.getNiveau().equals("PLATINUM")) {
        avantages.add("Logo sur affiches");
    }

    if (p.getNiveau().equals("SILVER") || p.getNiveau().equals("GOLD")
        || p.getNiveau().equals("PLATINUM")) {
        avantages.add("Stand aux événements");
    }

    // ... etc, de plus en plus illisible
}
```

Pourquoi c'est un problème ? - Duplication des conditions - Impossible à maintenir - Risque d'erreur élevé

Indice : Pensez héritage ou composition pour modéliser cette hiérarchie d'avantages.

Piège 6 : Pas de gestion d'erreurs

```
public void inscrireEvenement(String email, String nomEvenement) {
    Adherent a = trouverAdherent(email);
    Evenement e = trouverEvenement(nomEvenement);
    e.ajouterParticipant(a);
}
```

Que se passe-t-il si : - L'email n'existe pas ? → `AdherentIntrouvableException` - L'événement n'existe pas ? → `EvenementIntrouvableException` - L'adhérent est déjà inscrit ? → `InscriptionDupliqueeException` - Il n'y a plus de places ? → `CapaciteMaxAtteinteException`

À retenir : Les exceptions ne sont pas un bonus, elles sont **obligatoires** pour un code professionnel. Chaque méthode doit gérer ses cas d'erreur avec des exceptions personnalisées et des messages clairs.

Voici comment structurer vos exceptions :

```
// Exception de base pour le BDE (optionnel mais propre)
public abstract class BDEException extends RuntimeException {
    public BDEException(String message) { super(message); }
}

// Exceptions spécifiques
public class AdherentIntrouvableException extends BDEException {
    public AdherentIntrouvableException(String email) {
        super("Aucun adhérent trouvé avec l'email : " + email);
    }
}
```

```

public class InscriptionDupliqueeException extends BDEException {
    public InscriptionDupliqueeException(Adherent a, Evenement e) {
        super(a.getPrenom() + " est déjà inscrit à " + e.getNom());
    }
}

public class ArticleIndisponibleException extends BDEException {
    public ArticleIndisponibleException(String nom) {
        super("L'article " + nom + " n'est plus disponible");
    }
}

```

Pour aller plus loin

Niveau 1 — Pour ceux qui terminent rapidement

1.1 Historique des transactions Le trésorier veut un historique complet : - Chaque recette et dépense doit être horodatée - On veut pouvoir filtrer par période (ce mois-ci, cette année) - On veut voir les transactions par catégorie (cotisations, événements, boutique, partenariats)

1.2 Événements avec jauge Certains événements ont un nombre de places limité : - Ajouter une capacité maximale optionnelle aux événements - Refuser les inscriptions quand c'est complet - Gérer une liste d'attente

1.3 Adhésion avec date d'expiration Les cotisations ne sont plus "payées ou non" mais ont une date de validité : - La cotisation est valable un an - Un adhérent peut renouveler avant expiration - Les réductions ne s'appliquent que si l'adhésion est valide

Niveau 2 — Pour ceux qui vont très vite

2.1 Système de points de fidélité Les adhérents cumulent des points : - 1 point par euro dépensé (événements + boutique) - Les points peuvent être convertis en réduction (100 points = 5€) - Certains articles ne sont achetables qu'avec des points

2.2 Votes et élections Le BDE doit organiser des élections : - Créer des scrutins (élection bureau, votes sur événements...) - Seuls les adhérents à jour de cotisation peuvent voter - Plusieurs modes de scrutin (uninominal, proportionnel) - Afficher les résultats

2.3 Système de notifications Les adhérents peuvent s'abonner à des notifications : - Nouvel événement dans mes clubs - Places disponibles pour un événement complet - Nouvelle réduction partenaire - Cotisation bientôt expirée

Comment implémenter ce système sans coupler fortement toutes les classes ?

Indice : Pattern Observer

Niveau 3 — Pour les très rapides

3.1 Import/Export des données

- Exporter les adhérents au format CSV
- Exporter les finances au format compatible Excel
- Importer des adhérents depuis un fichier
- Générer un rapport PDF de l'activité du BDE

Comment faire pour que l'ajout d'un nouveau format d'export soit facile ?

3.2 Multi-BDE (Fédération) Votre application doit maintenant gérer plusieurs BDE : - Chaque école a son BDE - Un événement inter-BDE peut être organisé - Un adhérent d'un BDE peut s'inscrire à un événement d'un autre BDE (tarif "extérieur") - Chaque BDE a son propre budget

Cette évolution va-t-elle casser votre conception ?

3.3 Système de permissions Les utilisateurs de l'application ont des rôles différents : - Adhérent lambda : voir ses infos, s'inscrire aux événements - Responsable club : gérer son club - Membre du bureau : accès à tout son périmètre - Trésorier : accès aux finances - Admin : accès total

Comment implémenter ce système de permissions de manière extensible ?

Règles du jeu

Ce qui est INTERDIT

1. **Les switch/if géants sur les types**
 - Pas de `if (type.equals("CPE")) ... else if (type.equals("EXTERIEUR")) ...`
 - Pas de `switch(niveau) { case "BRONZE": ... case "SILVER": ... }`
 - Utilisez le polymorphisme !
2. **Les classes "sac de données"**
 - Une classe avec uniquement des getters/setters est suspecte
 - Les comportements doivent être dans les objets
3. **Le copier-coller**
 - Si vous copiez du code, c'est qu'il y a une abstraction à trouver
 - Demandez-vous : "Est-ce que je peux factoriser ?"
4. **Les attributs de type String pour représenter un type**
 - Pas de `String type = "CPE"` ou `String organisateur = "BDE"`
 - Utilisez des classes, des énumérations, ou des interfaces

Ce qui est OBLIGATOIRE

1. **Refactoriser quand le besoin évolue**
 - Avant de commencer une nouvelle étape, regardez si votre code actuel est adapté
 - N'hésitez pas à tout restructurer si nécessaire
 2. **Expliquer votre design à l'enseignant**
 - Avant de passer à l'étape suivante, expliquez vos choix de conception
 - Dessinez vos classes sur papier si besoin
 3. **Tester chaque fonctionnalité**
 - Ne passez pas à l'étape suivante si l'étape actuelle ne marche pas
 - Créez un jeu de test (quelques adhérents, quelques événements)
 4. **Gérer les erreurs avec des exceptions personnalisées**
 - Pas de `return null` silencieux
 - Créez des classes d'exception dédiées (`AdherentIntrouvableException`, `InscriptionDupliqueeException`, etc.)
 - Utilisez des messages clairs
 5. **Utiliser les concepts vus en cours**
 - `static` pour les compteurs d'ID et méthodes utilitaires
 - `record` pour les données immuables (transactions, résumés)
 - Pattern matching `instanceof` quand vous devez vérifier un type
 - Nommer les principes SOLID que vous appliquez
-